

Week-01 HandsOn Solution

❖ Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

- Code:

```
using System;

public class Logger{
    private static Logger? instance;
    private static readonly object lockObj = new object();

    private Logger() { }

    public static Logger Instance
    {
        get
        {
            lock (lockObj)
            {
                if (instance == null)
                    instance = new Logger();
                return instance;
            }
        }
    }

    public void Log(string message){
        Console.WriteLine("Log: " + message);
    }
}

public class Program{
    public static void Main()
    {
        Logger log1 = Logger.Instance;
        Logger log2 = Logger.Instance;
        log1.Log("First message");
        Console.WriteLine(log1 == log2);
    }
}
```

- Output:

```
Log: First message
True
```

Exercise 2: Implementing the Factory Method Pattern

- **Code:**

```
using System;

interface IDocument{
    void Open();
}

class WordDocument : IDocument{
    public void Open() => Console.WriteLine("Word document opened.");
}

class PdfDocument : IDocument{
    public void Open() => Console.WriteLine("PDF document opened.");
}

abstract class DocumentFactory{
    public abstract IDocument CreateDocument();
}

class WordFactory : DocumentFactory{
    public override IDocument CreateDocument() => new WordDocument();
}

class PdfFactory : DocumentFactory{
    public override IDocument CreateDocument() => new PdfDocument();
}

public class Program{
    public static void Main(){
        DocumentFactory factory = new PdfFactory();
        IDocument doc = factory.CreateDocument();
        doc.Open();
    }
}
```

- **Output:**

```
PDF document opened.
```